

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2024/2025

MAY EXAMINATION

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

SATURDAY, 31 MAY 2025

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Instructions to Candidates:

This question paper consists of FIVE (5) questions .

Answer **ALL** questions in **Section A** and **ONLY ONE(1)** question in **Section B**. Each question carries 25 marks.

Should a candidate answers more than ONE (1) question in section B, marks will only be awarded for the FIRST question in that section in the order the candidate submits the answers.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION A (Answer all questions)**

Q1. (a) Consider the standard neural network with the following settings in Table 1:

Table 1: Standard Neural Network

Layer	Setting	Details
Input	1 feature ($F_x = 1$)	Input: x
Hidden layer	2 neurons (with no bias) ReLU activation	First neuron: $z_1 = w_1 x$ (no bias) $a_1 = \max(z_1, 0)$ Second neuron: $z_2 = w_2 x$ (no bias) $a_2 = \max(z_2, 0)$
Output layer	1 neuron (with no bias) ReLU activation	First neuron: $z_3 = w_3 a_1 + w_4 a_2$ (no bias) $\hat{y} = a_3 = \max(z_3, 0)$

The model is trained with mean squared error (MSE) loss where:

$$J(\hat{y}, y) = \frac{1}{2} (y - \hat{y})^2 \quad \text{and} \quad \frac{\partial J}{\partial \hat{y}} = \hat{y} - y$$

Given the following training sample and initial weights and biases:

- Training sample: $x = 2.0, y = 1.5$
- Hidden layer: $w_1 = 0.8, w_2 = -0.5$
- Output layer: $w_3 = 0.7, w_4 = -0.5$

- Draw the computational graph by forward propagation. The graph should consist only of multiplication, addition, max and MSE nodes. Partition the graph into the hidden, output and cost layers. (6 marks)
 - Perform the forward pass. Show all steps. (3 marks)
 - Compute the loss using the mean squared error (MSE) loss. (2 marks)
 - Discuss how MSE impacts the training of the network when the dataset is imbalanced in terms of its input feature distribution. (3 marks)
 - Compute $\partial J / \partial w_1$ using backpropagation on top of your answer in Q1(a)(i). Show the calculations of the gradients for all signals along the path from J to w_1 . You may ignore all remaining signals. (6 marks)
- (b) Suppose that the model achieves 98% accuracy on the training set but only 70% accuracy on the test set.
- What issue is the model facing? Explain why this occurs. (3 marks)
 - Can data augmentation reduce the issue at Q1(b)(i). Justify your answer. (2 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Q2. Convolutional Neural Networks (CNNs) are a class of deep learning models for processing spatial data. Answer the following questions on CNN.

- (a) (i) How do convolutional layers differ from fully connected layers in terms of feature extraction quality, computational efficiency and receptive field. (6 marks)
- (ii) Object detection models (e.g., YOLO) is able to detect both small to large sized objects in an image. Why can they do so even though they primarily use small filters (e.g., 3×3) in their convolutional layers? (2 marks)

- (b) You are designing a CNN for an image classification task on a dataset of 128×128 RGB images. The network has the following layers:

Conv Layer: 32 filters (3×3 kernel size, stride 1, same padding)
Max Pooling (2×2 pool size, stride 2)
Conv Layer: 64 filters (3×3 kernel, stride 1, same padding)
Max Pooling (2×2 pool size, stride 2)
Flatten
Fully Connected Layer (1000 neurons)

- (i) What are the dimensions of the final output feature map of the second pooling layer (before Flatten) given one sample? (4 marks)
- (ii) Compute the number of trainable parameters in the network. (3 marks)
- (iii) Would any of the following changes affect the model size (i.e., the number of trainable parameters)? Explain conceptually why or why not for each scenario:
- Using higher resolution input (e.g., 256×256 instead of 128×128).
 - Reducing the number of output classes.
 - Increasing number of filters in Conv layer. (3 marks)
- (iv) Suppose you want to reduce the total number of trainable parameters in the CNN without sacrificing too much performance. Propose changes to the architecture and explain how each one would help achieve this goal. (3 marks)
- (v) If the network starts overfitting on a small dataset, what network design choices could you make in the CNN architecture to improve generalization? Provide **TWO (2)** solutions and explain how they help. (4 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

- Q3. (a) Explain the key difference between a Recurrent Neural Network (RNN) and a standard feedforward neural network. Why are RNNs more suitable for sequence data? (4 marks)
- (b) Why does RNN only have short-term memory? (4 marks)
- (c) RNN are trained using Backpropagation Through Time (BPTT). Explain how BPTT can limit the RNN's ability to capture long-term dependencies and impose constraints on its overall architecture. (5 marks)
- (d) Consider an RNN with one hidden layer, single neuron. The input to the network in each time step t is a scalar (i.e., $input_dim = 1$). The following equation describes the forward pass:

$$h^{<t>} = \tanh(h^{<t-1>}W_{hh} + x^{<t>}W_{hx} + b)$$

Given the following settings:

- input sequence ($T=3$) : $\mathbf{x} = [x^{<1>}, x^{<2>}, x^{<3>}] = [0.5, 1.5, 2.5]$
- initial hidden state : $h^{<0>} = 0$
- Parameters : $W_{hh} = 0.3, W_{hx} = 1.5, b = 0$

Calculate the output of the RNN layer for all time steps. (6 marks)

- (e) Suppose the following PyTorch configuration is used to define an RNN

```
rnn = nn.RNN(input_size=3, hidden_size=5, num_layers = 1,
             bidirectional = True, batch_first=True)
```

For batch size 32 and sequence of length 128, calculate the dimensions of:

- (i) the input tensor $\mathbf{X}$. (2 marks)
- (ii) the last hidden state $\mathbf{h}^{<T>}$. (2 marks)
- (iii) the output tensor $\mathbf{H}$. (2 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION B (Choose ONE question only)**

- Q4. LoRA (Low-Rank Adaptation) is a type of Parameter Efficient Fine-Tuning (PEFT) algorithm. LoRA adds an offset weight $\mathbf{W}_{\text{loa}} = \mathbf{W} + \frac{\alpha}{r} \Delta \mathbf{W}$. Answer the following questions.
- (a) Identify **TWO (2)** challenges of full fine-tuning when adapting a Large Language Model (LLM) to various downstream tasks. Explain how LoRA addresses these challenges. (6 marks)
 - (b) Given $\mathbf{W}$ with a shape of (4, 8), factorize the offset weight $\Delta \mathbf{W}$ into $\mathbf{AB}$ using a rank of $r = 2$ and explain why this helps LoRA to be more efficient compared to full fine-tuning. (5 marks)
 - (c) Explain how LoRA can help to reduce storage space when it is used to finetune Large Language Models for different types of task. (3 marks)
 - (d) Explain if LoRA can avoid catastrophic forgetting issue after fine-tuning. (3 marks)
 - (e) Explain how the choice of the rank (r) in LoRA impacts the balance between model performance and training efficiency. What if r is too low or too high? (4 marks)
 - (f) Suppose LoRA is used to fine-tune a CNN model for classification but the finetuned model is still experiencing overfitting. Propose **TWO (2)** adjustments to the LoRA implementation to make it more robust to overfitting. Your solution should focus exclusively on the design or configuration changes to the LoRA method itself, not on external techniques like data augmentation or regularization. (4 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Q5. (a) Figure 5.1 shows the structure of an encoder layer in the Transformer model.

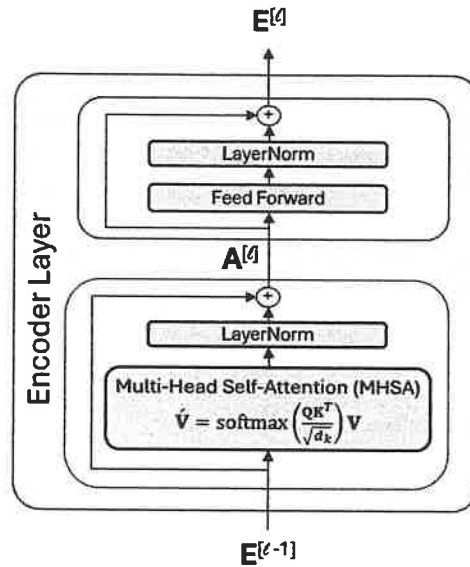


Figure 5.1: Encoder Layer in a Transformer

- (i) Identify the part of the MHSA equation that determines the parts of the sentence to pay attention to and explain how it works. (4 marks)
 - (ii) Explain how the attention mechanism enables the Transformer to capture long-range dependencies more effectively than traditional Recurrent Neural Networks (RNNs). (4 marks)
 - (iii) Identify the key components in the encoder layer that help mitigate the vanishing gradient problem. Explain how they address the issue. (4 marks)
 - (iv) Compare how transformer network embeds time information compared to RNN. Explain how this allows the transformer to have more layers without suffering from performance degradation. (4 marks)
- (b) Autoregressive Language Modeling (ARLM) is one of the common strategies to pretrain a Large Language Model.
- (i) Explain the concept of self-supervised learning and how it differs from supervised learning. (4 marks)
 - (ii) Describe how ARLM can be used for pre-training LLMs. In your answer, explain how the training set can be automatically generated and describe the training objective. (5 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Logistic Regression

Forward propagation

$$z = w_1 x_1 + \dots + w_{F_x} x_{F_x} + b = \mathbf{x}^T \mathbf{w} + b$$

$$\mathbf{z} = \mathbf{X} \mathbf{w} + b$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}}$$

Gradient Descent

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{1}{M} \left(\frac{1-y}{(1-\hat{y})} - \frac{y}{\hat{y}} \right)$$

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} * (\hat{y} * (1 - \hat{y}))$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{z}}$$

$$\frac{\partial J}{\partial b} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}} \right)^T$$

Standard Neural Network

Forward propagation

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$$

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \text{ (softmax)}$$

Gradient Descent

$$\frac{\partial J}{\partial \hat{\mathbf{y}}} = \frac{1}{C \times B} \left(\frac{1-\mathbf{y}}{1-\hat{\mathbf{y}}} - \frac{\mathbf{y}}{\hat{\mathbf{y}}} \right)$$

$$\frac{\partial J}{\partial \mathbf{Z}^{[l]}} = \begin{cases} \frac{\partial J}{\partial \mathbf{A}^{[l]}} * \mathbf{A}^{[l]} * (1 - \mathbf{A}^{[l]}) & \text{for sigmoid activation} \\ \frac{\partial J}{\partial \mathbf{A}^{[l]}} * (\mathbf{Z}^{[l]} > 0) & \text{for relu activation} \end{cases}$$

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \mathbf{A}^{[l-1]T} \frac{\partial J}{\partial \mathbf{Z}^{[l]}}$$

$$\frac{\partial J}{\partial \mathbf{b}^{[l]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \right)^T$$

$$\frac{\partial J}{\partial \mathbf{A}^{[l-1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \mathbf{W}^{[l]T}$$

Activation Functions

Sigmoid

$$g(z) = \frac{1}{1+e^{-z}}, \quad \frac{dg}{dz} = g(z) \cdot (1 - g(z))$$

Tanh

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \frac{dg}{dz} = 1 - g(z)^2$$

ReLU

$$g(z) = \max(0, z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Leaky ReLU

$$g(z) = \max(0.01z, z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0.01 & \text{if } z < 0 \end{cases}$$

Cost Function

Binary cross entropy

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) \\ = \text{mean}(-\mathbf{y} * \ln \hat{\mathbf{y}} - (1 - \mathbf{y}) * \ln(1 - \hat{\mathbf{y}}))$$

Binary cross entropy (multi-label)

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{C \times B} \sum_{c=1}^C \sum_{i=1}^B -y_c^{(i)} \ln \hat{y}_c^{(i)} - (1 - y_c^{(i)}) \ln(1 - \hat{y}_c^{(i)}) \\ = \text{mean}(-\mathbf{Y} * \ln \hat{\mathbf{Y}} - (1 - \mathbf{Y}) * \ln(1 - \hat{\mathbf{Y}}))$$

Negative Log Likelihood

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -\ln(p_c^{(i)})$$

RNN and LSTM

RNN

$$\mathbf{H}^{<t>} = g(\mathbf{H}^{<t-1>} \mathbf{W}_{hh} + \mathbf{X}^{<t>} \mathbf{W}_{hx} + \mathbf{b}^T)$$

$$\mathbf{H}^{<t>} = g([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_h + \mathbf{b}^T)$$

LSTM

$$\mathbf{F}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_f + \mathbf{b}_f^T)$$

$$\mathbf{I}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_i + \mathbf{b}_i^T)$$

$$\mathbf{O}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_o + \mathbf{b}_o^T)$$

$$\tilde{\mathbf{C}}^{<t>} = \tanh([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_c + \mathbf{b}_c^T)$$

$$\mathbf{C}^{<t>} = \mathbf{I}^{<t>} * \tilde{\mathbf{C}}^{<t>} + \mathbf{F}^{<t>} * \mathbf{C}^{<t-1>}$$

$$\mathbf{H}^{<t>} = \mathbf{O}^{<t>} * \tanh(\mathbf{C}^{<t>})$$

Convolutional Neural Network

$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

$$o = \left\lfloor \frac{i+2p-f}{s} + 1 \right\rfloor$$

$$\# \text{params in a filter} = c_f \times h_f \times w_f + 1$$

$$\# \text{params in a CONV layer} = \# \text{filters} \times \# \text{params in a filter}$$

$$\# \text{computations} = \# \text{output activations} \times \# \text{computations per activation} \\ = \text{output volume size} \times \# \text{params in a filter}$$

Efficient Net

$$\text{depth } d = \alpha^\phi, \text{ width } w = \beta^\phi, \text{ resolution } r = \gamma^\phi$$

$$\text{subject to: } \alpha, \beta^2, \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Transformers

$$\hat{\mathbf{V}} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\hat{\mathbf{V}}_m = \text{softmax}\left(\frac{\mathbf{QK}^T + \mathbf{M}}{\sqrt{d_k}}\right) \mathbf{V}$$

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Optimizers

Momentum

$$\mathbf{v}^{<t>} := \beta \mathbf{v}^{<t-1>} + (1 - \beta) \frac{\partial J^{<t>}}{\partial \mathbf{w}}$$

$$\mathbf{v}^{<t>} := \frac{1}{1 - \beta^t} \mathbf{v}^{<t>}$$

$$\mathbf{w}^{<t>} := \mathbf{w}^{<t-1>} - \alpha \mathbf{v}^{<t>}$$

$$\text{weight}^{<t-n>} = (1 - \beta) \beta^n$$

RMSProp

$$\mathbf{s}^{<t>} := \beta \mathbf{s}^{<t>} + (1 - \beta) \left(\frac{\partial J^{<t>}}{\partial \mathbf{w}} \right)^2$$

$$\mathbf{w}^{<t>} := \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\mathbf{s}^{<t>} + \epsilon}} \right) \frac{\partial J^{<t>}}{\partial \mathbf{w}}$$

Adam

$$\mathbf{v}^{<t>} := \beta_1 \mathbf{v}^{<t-1>} + (1 - \beta_1) \frac{\partial J^{<t>}}{\partial \mathbf{w}}$$

$$\tilde{\mathbf{v}}^{<t>} = \frac{\mathbf{v}^{<t>}}{(1 - \beta_1^t)}$$

$$\mathbf{s}^{<t>} := \beta_2 \mathbf{s}^{<t-1>} + (1 - \beta_2) \left(\frac{\partial J^{<t>}}{\partial \mathbf{w}} \right)^2$$

$$\tilde{\mathbf{s}}^{<t>} = \frac{\mathbf{s}^{<t>}}{(1 - \beta_2^t)}$$

$$\mathbf{w}^{<t>} = \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\tilde{\mathbf{s}}^{<t>} + \epsilon}} \right) \tilde{\mathbf{v}}^{<t>}$$

Learning Rate Decay

$$\alpha^{<t>} = \frac{1}{1 + \tau t} \alpha^{<0>}$$

$$\alpha^{<t>} = \gamma^{t/T} \alpha^{<0>}$$

$$\alpha^{<t>} = e^{-k \times t} \alpha^{<0>}$$

Initialization

Xavier Initialization

$$\text{var}(\mathbf{W}^{[l]}) = \frac{2}{(F^{[l-1]} + F^{[l]})}$$

Kaiming Initialization

$$\text{var}(\mathbf{W}^{[l]}) = \frac{2}{F^{[l-1]}}$$

Normalization

BatchNorm

$$\hat{\mathbf{Z}}[:, f] = \frac{\mathbf{Z}[:, f] - \mu_f}{\sqrt{v_f + \epsilon}}$$

$$\hat{\mathbf{Z}}[:, c, :, :] = \frac{\mathbf{Z}[:, c, :, :] - \mu_c}{\sqrt{v_c + \epsilon}}$$

$$\tilde{\mathbf{Z}}[:, c, :, :] = \gamma_c \hat{\mathbf{Z}}[:, c, :, :] + \beta_c$$

$$\mu_c^{(t)} = \text{mean}(\mathbf{Z}^{(t)}[:, c, :, :]) \quad (\text{training mode})$$

$$v_c^{(t)} = \text{variance}(\mathbf{Z}^{(t)}[:, c, :, :]) \quad (\text{training mode})$$

$$\tilde{\mu}_c = (1 - m) \tilde{\mu}_c + m \mu_c^{(t)} \quad (\text{eval mode})$$

$$\tilde{v}_c = (1 - m) \tilde{v}_c + m v_c^{(t)} \quad (\text{eval mode})$$

LayerNorm

$$\hat{\mathbf{Z}}[i, t, :] = \frac{(\mathbf{Z}[i, t, :] - \mu^{(i) <t>})}{\sqrt{v^{(i) <t>} + \epsilon}}$$

$$\tilde{\mathbf{Z}}[i, t, :] = \gamma^{(i) <t>} \hat{\mathbf{Z}}[i, t, :] + \beta^{(i) <t>}$$

$$\mu^{(i) <t>} = \text{mean}(\mathbf{Z}[i, t, :]) \quad (\text{training and eval model})$$

$$v^{(i) <t>} = \text{variance}(\mathbf{Z}[i, t, :]) \quad (\text{training and eval model})$$